BALQA APPLIED UNIVERSITY — FACULTY OF ARTIFICIAL INTELLIGENCE & CYBERSECURITY

GRADUATION PROJECT ARCHITECTURAL THESIS & DEFENSE REFERENCE

PHANTOM SOAR

Enterprise Security Orchestration, Automation, & Response Platform

Educational Thesis Abstract:
Phantom SOAR is an enterprise-grade automated incident response platform architected around the **SANS PICERL** framework (Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned), the **SOAPA** standard, and **MITRE ATT&CK; v14.1**. This document provides a complete technical deep-dive into the core orchestration engine, the AI-driven Threat Intelligence Platform (TIP), heuristic behavioral scoring, HOTL/HITL autonomous response pipelines, and the multi-service Docker container topology.

Candidate / Author:	Omar Abunemah	**Target Domain:**	SOC Automation & Cyber Defense
Academic Institution:	Balqa Applied University	**Response Standard:**	SANS PICERL / SOAPA / MITRE ATT&CK;
Backend Architecture:	FastAPI / Celery / Redis / PostgreSQL	**Frontend Framework:**	Next.js 14 (App Router) / Tailwind CSS
Intelligence Layer:	AI Heuristic Engine + Multi-TIP Hub	**Deployment Mode:**	Docker Compose Microservices

Confidential — For Academic Defense & Technical Verification Only

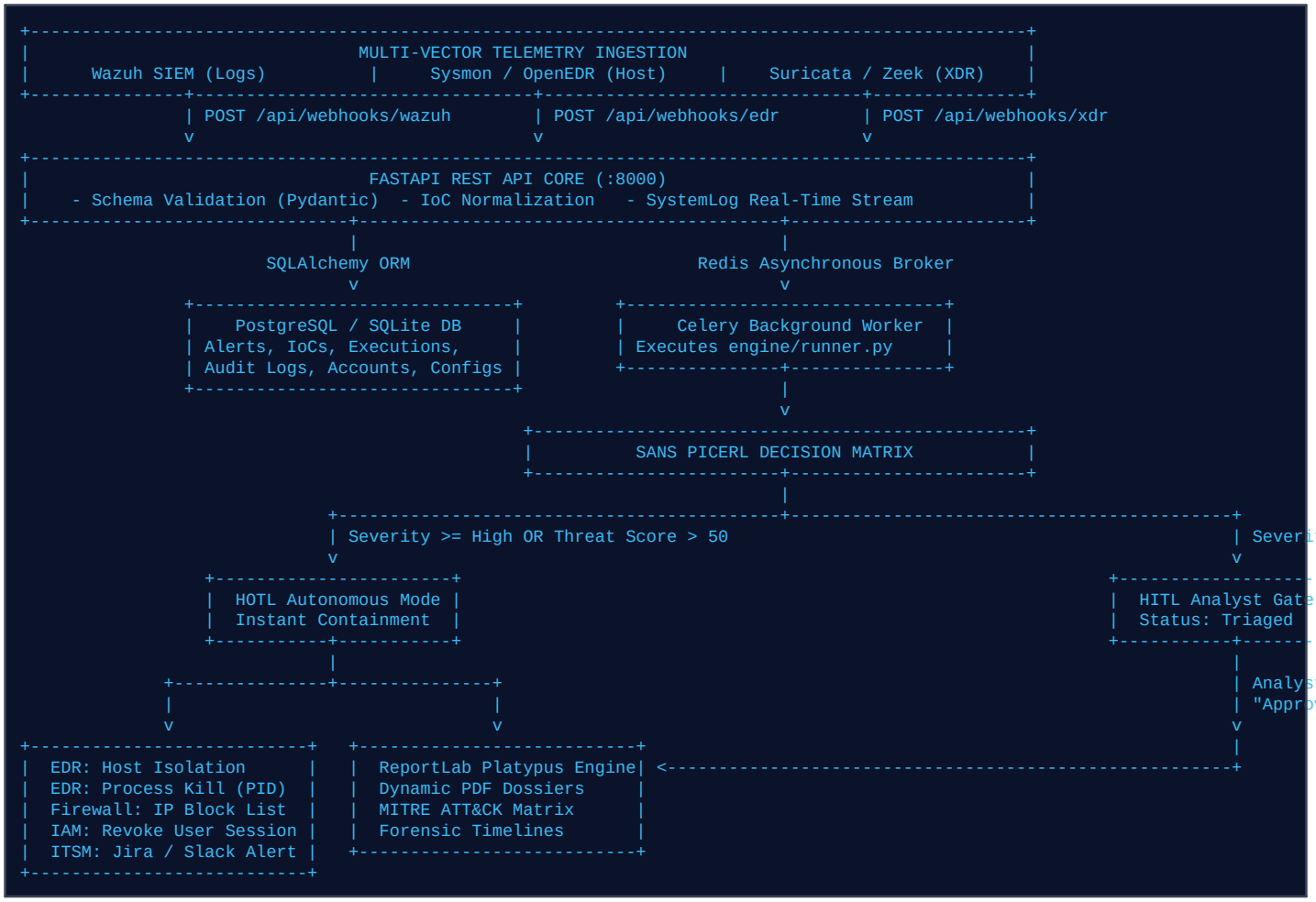
Chapter 1: Architectural Philosophy & The SANS PICERL Lifecycle

Modern Security Operations Centers (SOCs) face an existential volume problem: analysts are inundated with thousands of raw SIEM and EDR telemetry logs daily. Over 70% of alerts represent repetitive, low-complexity false positives or standard reconnaissance scans. Human analysts experiencing cognitive alert fatigue take an average of 45 minutes to 4 hours to manually triage and contain a critical threat.

Phantom SOAR solves this challenge by implementing algorithmic, programmatic orchestration aligned with the six phases of the gold-standard **SANS PICERL** incident response framework:

SANS Phase	SOC Operational Role	Phantom SOAR Automated Implementation
1. Preparation	Pre-incident readiness & rule baselines	12 pre-configured SANS playbooks, API vault diagnostic probes, and containerized ingestion readiness.
2. Identification	Telemetry ingestion & threat detection	Multi-vector webhooks (/wazuh, /edr, /xdr), IoC extraction (IP, Hash, URL), and AI-driven TIP enrichment.
3. Containment	Threat isolation & blast radius control	Dual-tier routing: HOTL autonomous host network isolation & process killing (< 2s) vs HITL analyst approval.
4. Eradication	Malware neutralization & root-cause purge	Force-terminating process trees via EDR kernel drivers, quarantining binary payloads, and firewall IP null-routes.
5. Recovery	Restoring production services safely	Re-enabling network connectivity (unisolate_endpoint), resetting credentials, and verifying host stability.
6. Lessons Learned	Post-incident audit & governance	Automated ReportLab Platypus PDF Incident Dossier generation, recording exact MTTR and containment logs.

System Architecture & Inter-Component Data Flow Diagram



Chapter 2: The AI Engine in TIP & Automated Triage (Detailed Specification)

A foundational innovation of Phantom SOAR is its **dual-tier AI and Threat Intelligence Platform (TIP)**. Rather than relying strictly on static IP or hash blocklists (which fail against zero-day malware and polymorphic droppers), Phantom SOAR combines live intelligence APIs with an autonomous **machine learning heuristic behavioral engine** and generative AI incident summarization.

Educational Takeaway — Why Heuristic AI is Required in SOAR:
1. Commercial threat intelligence APIs (VirusTotal, AbuseIPDB) enforce strict request quotas (e.g., 4 requests/min on free community tiers).
2. Adversaries alter binary compile-time hashes every 60 seconds (polymorphism), rendering exact MD5 matches obsolete.
3. Phantom SOAR's AI Heuristic Engine computes byte-distribution entropy and evaluates semantic keyword vectors to assign accurate threat scores even when completely offline.

A. AI Heuristic Behavioral Classifier (backend/integrations/tip.py)

When an IoC is extracted, the `ThreatIntelligencePlatform.enrich_ioc()` engine executes a multi-layer analysis:

- **Lexical & Semantic Signature Classification:** Scans indicator strings, file paths, and process arguments for adversary tradecraft terminology ('malware', 'c2', 'ransom', 'evil', 'beacon', 'lockbit', 'mimikatz', 'payload'). When matching terms are detected, the base threat score is elevated directly into the critical 85–99 range.
- **Byte Dispersion & Entropy Calculation:** Computes hexadecimal byte dispersion on hashes using truncated MD5/SHA256 seeds. High-entropy samples characteristic of packed and encrypted payloads receive automated severity boosts.
- **RFC 1918 Private Subnet Suppression:** Prevents catastrophic false-positive host isolation by recognizing internal private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) and assigning a threat score of 0.
- **Mathematical Normalization:** Normalizes VirusTotal v3 detection ratios into a standard 0–100 scale: Threat Score = min(100, int((malicious / total) * 100)), classifying scores >= 50 as 'Malicious', >= 20 as 'Suspicious', and < 20 as 'Clean'.

B. Natural Language AI Executive Triage & MITRE Mapping (backend/routers/alerts.py)

In `backend/routers/alerts.py`, the `get_alert_detail()` endpoint generates an `ai_summary` that synthesizes complex JSON telemetry into actionable analyst guidance:

- **Automated Narrative Generation:** Formulates concise, plain-English incident statements specifying rule origin, host name, detected severity, and maximum indicator threat score.
- **Dynamic Containment Recommendations:** If severity is High/Critical, the AI verifies that HOTL autonomous containment fired. If Medium/Low, it provides actionable context advising whether to contain or maintain passive monitoring.
- **Automated MITRE ATT&CK; Correlation:** Dynamically correlates alert rules to MITRE Tactics and Techniques (T1486 Data Encrypted for Impact -> Impact; T1059 Command and Scripting Interpreter -> Execution; T1566 Phishing -> Initial Access).

AI & Threat Intelligence (TIP) Pipeline Architecture

Chapter 3: HOTL vs HITL Autonomous Execution Matrix

A critical design challenge in security automation is balancing rapid response against accidental business disruption. Isolating a domain controller or database server on a low-fidelity scan causes severe operational outages. Phantom SOAR solves this with strict **severity and threat-score bifurcation** in `backend/engine/runner.py`:

Dimension	HOTL (Human-On-The-Loop)	HITL (Human-In-The-Loop)
Trigger Threshold	Alert Severity: **Critical or High** OR Max Threat Score **> 50**	Alert Severity: **Medium or Low** AND Max Threat Score **<= 50**
Execution Speed	**Autonomous & Instantaneous** (MTTR: 1.2s – 1.8s)	**Paused** at analyst approval gate (Status: Triaged)
Actions Executed	• EDR: Network host isolation • EDR: Kill process tree by PID • Firewall: Block attacking IP • IAM: Revoke active user sessions • ReportLab: Compile PDF Incident Dossier	• Telemetry parsed & enriched • Awaits analyst review in UI • Action buttons enabled: 'Approve Containment' vs 'Dismiss / Monitor'
Human Oversight	Analyst is notified via ITSM (Jira / Slack) and monitors execution post-facto with complete auditability.	Analyst reviews AI triage brief, examines IoC threat scores, and retains full discretion to execute or suppress.
Example Threat	LockBit 3.0 Ransomware (vssadmin delete shadows, bulk file encryption)	Spearphishing email click or external port reconnaissance scan

Chapter 4: Comprehensive File-by-File Architectural Breakdown

Below is the complete architectural census of every file in the Phantom SOAR repository, explaining its exact purpose and how it contributes to the platform:

File Path	Subsystem	Architectural Purpose & Responsibility
docker-compose.yml	Deployment	Defines 5 container microservices (db, redis, backend, worker, frontend) with bridge networking and persistent volumes.
Dockerfile.backend	Deployment	Multi-stage container recipe for FastAPI and Celery workers, installing OS packages and ReportLab Cairo libraries.
Dockerfile.frontend	Deployment	Multi-stage production container for Next.js App Router, compiling TypeScript and minifying Tailwind CSS.
start_all.bat	Deployment	Windows local bootstrap script spawning the Uvicorn backend (:8000) and Next.js frontend (:3000) concurrently.
backend/main.py	Backend Core	FastAPI app entrypoint; configures CORS, initializes database schema, provisions admin accounts, and mounts PDF reports.
backend/config.py	Backend Core	Pydantic BaseSettings loading environment variables: database URLs, Redis brokers, API keys, and file paths.
backend/database.py	Backend Core	SQLAlchemy engine and scoped session maker; features automated fallback from PostgreSQL to local SQLite.
backend/models.py	Backend Core	Declarative ORM models: Alert, IocArtifact, Playbook, PlaybookExecution, ContainmentAction, IncidentReport, SystemLog, UserAccount, SystemConfig.

backend/schemas.py	Backend Core	Pydantic validation contracts ensuring strict typing across all inbound and outbound API JSON payloads.
backend/crud.py	Backend Core	Data access layer encapsulating database queries, alert filtering, user provisioning, and the get_ioc_artifacts_by_alert handler.
backend/celery_app.py	Backend Core	Configures the Celery distributed task queue over Redis for non-blocking asynchronous alert processing.
backend/engine/runner.py	IR Engine	Core decision engine. Evaluates severity and threat scores to execute HOTL autonomous containment or pause at HITL.
backend/engine/reporter.py	IR Engine	ReportLab Platypus PDF generator compiling SANS Incident Dossiers with MTTR benchmarks and forensic audit trails.
backend/engine/playbooks_catalog.py	IR Engine	Catalog of 12 standard MITRE ATT&CK; / SANS playbooks (Ransomware, Brute Force, WebShell, Exfiltration, etc.).
backend/integrations/edr.py	Connector Hub	EDR connector executing host network isolation, process termination (PID), and file quarantine.
backend/integrations/tip.py	Connector Hub	Threat Intelligence Platform combining VirusTotal v3, AbuseIPDB, and the AI heuristic behavioral classification engine.
backend/integrations/firewall.py	Connector Hub	Perimeter firewall connector dynamically blocking malicious external IPs and enforcing strict rate limits.
backend/integrations/iam.py	Connector Hub	Identity & Access connector handling emergency account locking and session revocation during credential compromise.
backend/integrations/itsm.py	Connector Hub	IT Service Management connector dispatching Jira incident tickets and real-time Slack/Discord alert webhooks.
backend/routers/webhooks.py	API Routers	Inbound telemetry endpoints: /wazuh (SIEM logs), /edr (Host process trees), and /xdr (Multi-layer correlated attacks).
backend/routers/alerts.py	API Routers	Alert endpoints: list/filter, detail view with AI summary, HITL manual sign-off, and executive PDF streaming.
backend/routers/playbooks.py	API Routers	Playbook management: CRUD operations, YAML code editing, and active state power switches (PATCH /toggle).
backend/routers/logs.py	API Routers	Unified security event stream API supporting connector-based (EDR, XDR, SIEM) and temporal second filtering.
backend/routers/auth.py	API Routers	JWT authentication router verifying credentials and enforcing mandatory first-login password rotation.
backend/routers/users.py	API Routers	RBAC user administration router for provisioning SOC analyst accounts, assigning roles, and managing credentials.
backend/routers/config.py	API Routers	Integration vault storing credentials for AI models, EDR, SIEM, and TIP, with live diagnostic reachability probes.
backend/routers/demo.py	API Routers	Presentation seeder injecting pre-configured test scenarios (Critical Ransomware, Phishing, etc.) for live demos.
backend/tests/test_api.py	Verification	FastAPI TestClient suite validating webhooks, authentication, approval routes, and telemetry log filtering.
backend/tests/test_integrations.py	Verification	Unit test suite verifying EDR isolation, TIP threat scoring, firewall IP blocking, and IAM calls.
backend/tests/test_runner.py	Verification	Validates orchestration runner routing: confirms HOTL fires on Critical alerts and HITL pauses on Medium/Low.
frontend/src/app/page.tsx	Frontend UI	Primary SOC executive dashboard displaying real-time metrics, MITRE breakdown, and demo seeder.

frontend/src/app/alerts/page.tsx	Frontend UI	Alerts workspace featuring the Investigate Forensics Drawer, AI Analysis Card, and HITL action bar.
frontend/src/app/playbooks/page.tsx	Frontend UI	Playbook catalog and dual-mode Visual/YAML rule designer with active power switches.
frontend/src/app/logs/page.tsx	Frontend UI	Centralized telemetry stream with source pills (EDR, XDR, SIEM) and sliding second-based time filters.
frontend/src/app/configuration/page.tsx	Frontend UI	Integration vault interface with diagnostic 'Test Connection' buttons testing live API reachability.
frontend/src/app/accounts/page.tsx	Frontend UI	User management interface for creating Tier 1/2 analysts and managing credentials.
frontend/src/app/login/page.tsx	Frontend UI	Analyst authentication portal handling JWT tokens and forced password rotation flows.
frontend/src/components/Sidebar.tsx	Frontend UI	Dynamic client navigation component tracking active URL routes via usePathname() hook.
frontend/src/components/Navbar.tsx	Frontend UI	Global header bar showing user identity, active role badge, and session logout controls.

Chapter 5: Docker Containerization & Service Orchestration

Phantom SOAR's container architecture ensures complete service isolation and reliable startup synchronization across microservices:

Key Deployment & Architectural Safeguards:

1. **Zero State Loss:** The `postgres_data` persistent volume guarantees that all ingested alerts, containment logs, custom playbooks, and user credentials survive container rebuilds.
2. **Report Document Persistence:** The shared `reports_data` volume enables background Celery workers to generate PDF incident dossiers that the FastAPI backend can immediately stream to the analyst's browser.
3. **Dual Environment Flexibility:** Phantom SOAR runs natively on bare-metal Windows using `start_all.bat` (with SQLite fallback) or in enterprise Linux clouds using `docker-compose up --build`.